เอกสารสรุป Controller

ระบบคำนวณภาษีเงินได้บุคคลธรรมดา
Laravel - Blade + Controller

ขอบเขตเอกสาร

ตรวจจากไฟล์ Controller และ Route ที่มีอยู่ในโปรเจกต์ ณ วันที่ 15 ตุลาคม 2026 อธิบายชื่อฟังก์ชัน พารามิเตอร์ Route ขั้นตอนทำงาน Validation สิทธิ์เข้าถึง การอ่าน/บันทึกฐานข้อมูล และผลลัพธ์ของแต่ละฟังก์ชัน

Controller ทั้งหมด	8 ไฟล์
Controller ที่มีฟังก์ชันงานระบบ	7 ไฟล์
ฟังก์ชันทั้งหมด	45 ฟังก์ชัน
Controller แม่	Controller.php ไม่มีฟังก์ชันเฉพาะของโปรเจกต์

หมายเหตุ: เอกสารนี้อธิบายพฤติกรรมตามโค้ดปัจจุบัน ไม่ใช่เอกสารยืนยันความถูกต้องทางกฎหมายภาษี

ภาพรวม Controller

Controller	หน้าที่หลัก	จำนวน
HomeController	ควบคุมหน้าสาธารณะหลักของเว็บไซต์ ไม่มีการอ่านหรือแก้ไขฐานข้อมูล	**3**
AuthController	จัดการ Login, Register และ Logout รวมถึง rate limit, session และการรองรับ Two-factor authentication	**5**
MemberController	ควบคุม Dashboard ของสมาชิก	**1**
ProfileController	แสดงและแก้ไขชื่อ อีเมล และรหัสผ่านของผู้ใช้ปัจจุบันผ่าน Blade + Controller	**2**
TaxCalculationController	รับข้อมูลจากแบบฟอร์ม คำนวณภาษีตามข้อมูลอ้างอิงในฐานข้อมูล เก็บผลล่าสุดใน session และบันทึกเป็นประวัติ	**4**
TaxRecordController	แสดงและจัดการประวัติผลคำนวณ รวมถึงเพิ่ม/แก้ไข/ลบรายการเงินได้ ค่าใช้จ่าย และค่าลดหย่อน พร้อมคำนวณยอดใหม่ทุกครั้ง	**13**
AdminController	Dashboard ผู้ดูแล จัดการผู้ใช้ และจัดการข้อมูลอ้างอิงทีสูตรภาษีใช้จากฐานข้อมูล ทุก Route อยู่ใต้ middleware auth + admin	**17**
Controller (Base)	คลาสแม่ของ Controller อืน ไม่มีฟังก์ชันเฉพาะในโปรเจกต์	**0**

เส้นทางการทำงานหลัก

- ผู้ใช้กรอกแบบฟอร์ม -> TaxCalculationController::calculateTax() ตรวจข้อมูลและคำนวณ -> เก็บผลไว้ใน session
- สมาชิกกดบันทึก -> TaxCalculationController::save() สร้าง TaxRecord พร้อม Income, Expense และ Deduction
- สมาชิกเปิดประวัติ -> TaxRecordController แสดงและแก้รายการย่อย -> calculateRecord() คำนวณผลรวมใหม่
- Admin แก้ข้อมูลอ้างอิง -> AdminController บันทึก IncomeType, ExpenseType, DeductionType และ TaxBracket -> สูตรอ่านค่าใหม่จากฐานข้อมูล

HomeController

app/Http/Controllers/HomeController.php

หน้าที	ควบคุมหน้าสาธารณะหลักของเว็บไซต์ ไม่มีการอ่านหรือแก้ไขฐานข้อมูล
จำนวนฟังก์ชัน	3

1. index()

ตำแหน่ง	บรรทัด 9
Route	**GET / | name: home**
หน้าที	เปิดหน้าแรกของเว็บไซต์

ขั้นตอนการทำงาน

- เรียก Blade view ชื่อ index

ผลลัพธ์

- คืนค่า HTML ของหน้า resources/views/index.blade.php

2. about()

ตำแหน่ง	บรรทัด 13
Route	**GET /about**
หน้าที	เปิดหน้าเกี่ยวกับโครงการ

ขั้นตอนการทำงาน

- เรียก Blade view ชื่อ about

ผลลัพธ์

- คืนค่า HTML ของหน้า resources/views/about.blade.php

3. knowledge()

ตำแหน่ง	บรรทัด 17
Route	**GET /knowledge**
หน้าที	เปิดหน้าความรู้เกี่ยวกับภาษี

ขั้นตอนการทำงาน

- เรียก Blade view ชื่อ knowledge

ผลลัพธ์

- คืนค่า HTML ของหน้า resources/views/knowledge.blade.php

AuthController

app/Http/Controllers/AuthController.php

หน้าที	จัดการ Login, Register และ Logout รวมถึง rate limit, session และการรองรับ Two-factor authentication
จำนวนฟังก์ชัน	5

4. login(): View

ตำแหน่ง	บรรทัด 17
Route	**GET /login	name: login	middleware: guest**
หน้าที	แสดงแบบฟอร์มเข้าสู่ระบบ

ขั้นตอนการทำงาน

- เรียก view Auth.login

ผลลัพธ์

- คืนหน้า Login ให้ผู้ทียังไม่ได้เข้าสู่ระบบ

5. authenticate(Request $request): RedirectResponse

ตำแหน่ง	บรรทัด 22
Route	**POST /login และ POST /login/fortify	middleware: guest**
หน้าที	ตรวจข้อมูลและเข้าสู่ระบบให้ผู้ใช้

ขั้นตอนการทำงาน

- แปลงอีเมลเป็นตัวพิมพ์เล็กและตัดช่องว่างหัวท้าย
- ตรวจรูปแบบอีเมล รหัสผ่าน และค่า remember
- สร้าง rate-limit key จากอีเมลและ IP
- ถ้าบัญชีเปิด 2FA และรหัสผ่านถูกต้อง จะเก็บ login.id กับ login.remember ใน session แล้วส่งไปหน้า two-factor.login
- กรณีทั่วไปใช้ Auth::attempt() และ regenerate session หลังสำเร็จ
- เมื่อรหัสผ่านผิด บันทึกความพยายามไว้ 60 วินาที

การตรวจสอบ/ข้อจำกัด

- อีเมลและ IP เดิมผิดได้สูงสุด 5 ครั้งใน 1 นาที
- อีเมลและรหัสผ่านยาวไม่เกิน 255 ตัวอักษร
- เก็บค่าอีเมลเดิมไว้เมื่อ validation หรือ login ไม่ผ่าน

ผลลัพธ์

- สำเร็จ: กลับ URL ที่ผู้ใช้ตั้งใจเข้าหรือหน้า home พร้อมข้อความสำเร็จ
- ต้องใช้ 2FA: ไป route two-factor.login
- ล้มเหลว: กลับฟอร์มพร้อมข้อความผิดพลาด

6. register(): View

ตำแหน่ง	บรรทัด 92
Route	**GET /register	name: register	middleware: guest**
หน้าที	แสดงแบบฟอร์มสมัครสมาชิก

ขั้นตอนการทำงาน

- เรียก view Auth.register

ผลลัพธ์

- คืนหน้า Register

7. store(Request $request): RedirectResponse

ตำแหน่ง	บรรทัด 97
Route	**POST /register | name: register.store | middleware: guest, throttle:10,1**
หน้าที	สร้างบัญชีผู้ใช้ใหม่และเข้าสู่ระบบทันที

ขั้นตอนการทำงาน

- ทำอีเมลให้เป็นตัวพิมพ์เล็ก
- ตรวจชื่อ อีเมล รหัสผ่าน และ password_confirmation
- สร้าง User ใหม่โดยกำหนด role เป็น user เสมอ
- เข้ารหัสรหัสผ่านด้วย Hash::make() ก่อนบันทึก
- ใช้ Auth::login() แล้ว regenerate session

การตรวจสอบ/ข้อจำกัด

- อีเมลต้องไม่ซ้ำในตาราง users
- รหัสผ่านอย่างน้อย 8 ตัวอักษรและต้องตรงกับช่องยืนยัน
- Route จำกัดการสมัคร 10 ครั้งต่อนาที

ผลลัพธ์

- สร้างข้อมูล users 1 รายการ
- เข้าสู่ระบบและ redirect ไปหน้า home พร้อม flash message

8. logout(Request $request): RedirectResponse

ตำแหน่ง	บรรทัด 142
Route	**POST /logout | name: logout | middleware: auth**
หน้าที	ออกจากระบบอย่างปลอดภัย

ขั้นตอนการทำงาน

- เรียก Auth::logout()
- ทำลาย session เดิมด้วย invalidate()
- สร้าง CSRF token ใหม่ด้วย regenerateToken()

ผลลัพธ์

- redirect ไปหน้า home พร้อมข้อความออกจากระบบสำเร็จ

MemberController

app/Http/Controllers/MemberController.php

หน้าที	ควบคุม Dashboard ของสมาชิก
จำนวนฟังก์ชัน	1

9. index(Request $request)

ตำแหน่ง	บรรทัด 9
Route	**GET /member และ GET /dashboard | middleware: auth, member**
หน้าที	แสดงหน้าสมาชิกตามผู้ใช้ทีเข้าสู่ระบบ

ขั้นตอนการทำงาน

- อ่านผู้ใช้ปัจจุบันจาก $request->user()
- ส่งตัวแปร user ไปยัง view member.index

ผลลัพธ์

- คืนหน้า Dashboard ของสมาชิก

ProfileController

app/Http/Controllers/ProfileController.php

หน้าที	แสดงและแก้ไขชื่อ อีเมล และรหัสผ่านของผู้ใช้ปัจจุบันผ่าน Blade + Controller
จำนวนฟังก์ชัน	2

10. edit(Request $request)

ตำแหน่ง	บรรทัด 10
Route	**GET /member/profile | name: profile.edit | middleware: auth, member**
หน้าที	แสดงแบบฟอร์มแก้ไขข้อมูลส่วนตัว

ขั้นตอนการทำงาน

- อ่านผู้ใช้ที่เข้าสู่ระบบ
- ส่งตัวแปร user ไป view member.profile

ผลลัพธ์

- คืนหน้าแก้ไขโปรไฟล์

11. update(Request $request)

ตำแหน่ง	บรรทัด 16
Route	**PUT /member/profile | name: profile.update | middleware: auth, member**
หน้าที	บันทึกข้อมูลโปรไฟล์ที่แก้ไข

ขั้นตอนการทำงาน

- ตรวจชื่อ อีเมล และรหัสผ่านใหม่
- ปรับอีเมลเป็นตัวพิมพ์เล็กและตัดช่องว่าง
- เปลี่ยนรหัสผ่านเฉพาะเมื่อกรอกค่า โดยเข้ารหัสด้วย Hash::make()
- บันทึก User

การตรวจสอบ/ข้อจำกัด

- ชื่อและอีเมลจำเป็น
- อีเมลต้องไม่ซ้ำ โดยยกเว้น id ของผู้ใช้ปัจจุบัน
- รหัสผ่านเป็น optional แต่ถ้ากรอกต้องยาวอย่างน้อย 8 ตัวและยืนยันตรงกัน

ผลลัพธ์

- redirect กลับ profile.edit พร้อมข้อความสำเร็จ

TaxCalculationController

app/Http/Controllers/TaxCalculationController.php

หน้าที	รับข้อมูลจากแบบฟอร์ม คำนวณภาษีตามข้อมูลอ้างอิงในฐานข้อมูล เก็บผลล่าสุดใน session และบันทึกเป็นประวัติ
จำนวนฟังก์ชัน	4

12. show()

ตำแหน่ง	บรรทัด 18
Route	**GET /TaxCalculate | name: tax.show**
หน้าที	เปิดแบบฟอร์มคำนวณภาษี

ขั้นตอนการทำงาน

- สร้าง form เป็น array ว่าง
- อ่าน IncomeType ทีเปิดใช้งาน เรียงตาม code และ keyBy(code)
- ส่ง form กับ incomeTypes ไป view calculate

ผลลัพธ์

- คืนหน้าคำนวณภาษีพร้อมประเภทเงินได้ที Admin เปิดใช้งาน

13. calculateStepTax(float $netIncome, int $taxYear = 2569)

ตำแหน่ง	บรรทัด 25
Route	**ไม่มี Route โดยตรง - เรียกใช้ภายใน Controller**
หน้าที	คำนวณภาษีแบบขั้นบันไดจากตาราง tax_brackets

ขั้นตอนการทำงาน

- อ่านขั้นภาษีของปีทีระบุ เรียงจาก lower_bound
- แต่ละขั้นจะคำนวณเฉพาะส่วนของเงินได้สุทธิทีเกินขอบล่าง
- ขั้นสุดท้ายที upper_bound เป็น null ใช้เงินได้สุทธิเป็นขอบบน
- นำฐานภาษีในแต่ละช่วงคูณ rate แล้วรวมเป็นภาษีทั้งหมด

การตรวจสอบ/ข้อจำกัด

- ถ้าปีนันไม่มีขันภาษี ฟังก์ชันจะคืน 0; calculateTax ตรวจกรณีนีก่อนเรียก

ผลลัพธ์

- คืนตัวเลขภาษีขันบันไดชนิด float

14. calculateTax(Request $request)

ตำแหน่ง	บรรทัด 41
Route	**POST /TaxCalculate | name: tax.calculate**
หน้าที	ฟังก์ชันหลักสำหรับตรวจแบบฟอร์มและคำนวณภาษีเงินได้บุคคลธรรมดา

ขั้นตอนการทำงาน

- แปลงช่องตัวเลขว่าง 21 ช่องให้เป็น 0
- ตรวจช่วงตัวเลข ปีภาษี สถานภาพ ตัวเลือกเงินได้ วิธีหักค่าใช้จ่าย และ checkbox ยืนยันสิทธิ
- ตรวจว่า IncomeType ทีมีรายได้ยังเปิดใช้งาน และบังคับเลือกประเภทย่อยของ ม.40(5), 40(6), 40(8)
- ตรวจค่าใช้จ่ายจริงไม่เกินรายได้ ตรวจเงือนไขบุตร คนพิการ ผู้สูงอายุ/คนพิการ และข้อมูลอ้างอิงของปีภาษี

- รวมรายได้ ม.40(1)-40(8) และหักเงินได้ยกเว้นสูงสุด 190,000 บาทจากประเภทที่ผู้ใช้เลือก
- คำนวณค่าใช้จ่าย: ม.40(1)+(2) ใช้อัตราและเพดานร่วม, ม.40(3) สิทธิ, ม.40(4) ไม่หัก, ม.40(5) ค่าเช่า, ม.40(6) วิชาชีพ, ม.40(7) รับเหมา, ม.40(8) ธุรกิจ
- คำนวณค่าลดหย่อนส่วนตัว คู่สมรส บุตร บุตรเพิ่ม บุตรบุญธรรม อุปการะคนพิการ ประกันสังคม ชีวิต และสุขภาพ
- จำกัดเงินบริจาคทั่วไปไม่เกิน 10% ของยอดหลังหักค่าใช้จ่ายและค่าลดหย่อนอื่น
- คำนวณภาษีขั้นบันได และภาษีขั้นต่ำ 0.5% ของเงินได้ที่ไม่ใช่ ม.40(1) เมื่อยอดถึง 120,000 บาท
- เลือกภาษีขั้นต่ำเฉพาะเมื่อมากกว่า 5,000 บาทและมากกว่าภาษีขั้นบันได
- หักภาษี ณ ที่จ่ายและภาษีครึ่งปีเพื่อหายอดชำระเพิ่มหรือยอดคืน
- เก็บ form, ผลรวม, expense_data และ deduction_data ใน session key tax_calculation

การตรวจสอบ/ข้อจำกัด

- ตัวเลขเงินได้/ค่าใช้จ่าย/ลดหย่อน/ภาษีต้องอยู่ระหว่าง 0 ถึง 1 ล้านล้าน
- ปีภาษีรองรับ 2568 หรือ 2569
- กิจการอื่นใน ม.40(8) ต้องใช้ค่าใช้จ่ายจริง; การหักเหมาต้องยืนยันว่ารายได้ตรงกับกิจการ
- บุตรที่มีสิทธิเพิ่มต้องไม่มากกว่าบุตรชอบด้วยกฎหมาย; เมื่อใช้บุตรบุญธรรม จำนวนรวมต้องไม่เกิน 3
- ประกันสังคมต้องไม่เกินเพดานใน deduction_types
- ถ้าไม่มีข้อมูลค่าลดหย่อนหลักหรือขั้นภาษีของปีนั้น ระบบส่ง ValidationException

ผลลัพธ์

- คืน view calculate พร้อมผลคำนวณและข้อมูลฟอร์มเดิม
- ผลล่าสุดพร้อมบันทึกจะอยู่ใน session จนกว่าจะบันทึกหรือ session หมดอายุ

15. save(Request $request)

ตำแหน่ง	บรรทัด 418
Route	**POST /TaxCalculate/save | name: tax.save | middleware: auth, member**
หน้าที	บันทึกผลคำนวณล่าสุดเป็นประวัติพร้อมรายการย่อย

ขั้นตอนการทำงาน

- ตรวจชื่อประวัติ title สูงสุด 100 ตัวอักษร
- อ่าน tax_calculation จาก session; ถ้าไม่มีให้กลับไปคำนวณก่อน
- สร้าง TaxRecord พร้อมข้อมูลปี สถานภาพ รายละเอียดส่วนตัว ยอดรวม ภาษีชำระแล้ว ยอดเพิ่ม/คืน และ calculation_version = 2
- วน ม.40(1)-40(8) และสร้าง Income เฉพาะยอดที่มากกว่า 0
- สร้าง Expense แยกตาม Income โดยผูก ExpenseType ตามรหัสและปีภาษี
- สร้าง Deduction แยกตาม code; เงินบริจาคเก็บยอดกรอกใน amount และยอดที่ใช้ได้จริงใน allowed_amount
- ลบ tax_calculation ออกจาก session

การตรวจสอบ/ข้อจำกัด

- ผู้ใช้ต้องผ่าน auth และ member
- ต้องมีผลคำนวณใน session
- หากไม่กรอก title ระบบตั้งชื่อ ผลคำนวณภาษีปี [ปีภาษี]

ผลลัพธ์

- สร้าง tax_records, incomes, expenses และ deductions ที่สัมพันธ์กัน
- redirect ไปหน้ารายละเอียดประวัติพร้อมข้อความสำเร็จ

TaxRecordController

app/Http/Controllers/TaxRecordController.php

หน้าที	แสดงและจัดการประวัติผลคำนวณ รวมถึงเพิ่ม/แก้ไข/ลบรายการเงินได้ ค่าใช้จ่าย และค่าลดหย่อน พร้อมคำนวณยอดใหม่ทุกครั้ง
จำนวนฟังก์ชัน	13

16. index(Request $request)

ตำแหน่ง	บรรทัด 17
Route	**GET /member/tax-records | name: tax.records.index**
หน้าที	แสดงรายการประวัติของผู้ใช้ปัจจุบัน

ขั้นตอนการทำงาน

- ใช้ relationship taxRecords() ของ User
- เรียงรายการล่าสุดก่อนและแบ่งหน้าละ 10 รายการ

ผลลัพธ์

- คืน view member.tax-records พร้อมตัวแปร records

17. show(Request $request, TaxRecord $record)

ตำแหน่ง	บรรทัด 23
Route	**GET /member/tax-records/{record} | name: tax.records.show**
หน้าที	แสดงรายละเอียดประวัติและข้อมูลสำหรับแบบฟอร์มแก้ไข

ขั้นตอนการทำงาน

- ตรวจเจ้าของ TaxRecord
- eager load incomes, incomeType, expenses, expenseType, deductions และ deductionType
- อ่านประเภทเงินได้ที่ active และประเภทค่าใช้จ่าย/ลดหย่อนที่ active ของปีใน record

การตรวจสอบ/ข้อจำกัด

- ถ้า record ไม่ใช่ของผู้ใช้ปัจจุบัน ตอบ HTTP 403

ผลลัพธ์

- คืน view member.tax-record-show พร้อม record และรายการประเภทอ้างอิง

18. storeIncome(Request $request, TaxRecord $record)

ตำแหน่ง	บรรทัด 33
Route	**POST /member/tax-records/{record}/incomes**
หน้าที	เพิ่มรายการเงินได้ในประวัติ

ขั้นตอนการทำงาน

- ตรวจเจ้าของ record
- ตรวจ income_type_id และ amount
- คัดลอกชื่อ IncomeType มาไว้ใน type_name
- บันทึก Income แล้วเรียก calculateRecord()

การตรวจสอบ/ข้อจำกัด

- income_type_id ต้องมีจริง

- amount ต้องเป็นตัวเลขตั้งแต่ 0 ขึ้นไป

ผลลัพธ์

- เพิ่ม Income และปรับยอดภาษีของ record ใหม่

19. updateIncome(Request $request, TaxRecord $record, Income $income)

ตำแหน่ง	บรรทัด 48
Route	**PUT /member/tax-records/{record}/incomes/{income}**
หน้าที	แก้ไขประเภทและยอดเงินได้

ขั้นตอนการทำงาน

- ตรวจเจ้าของ record และตรวจว่า income อยู่ใน record
- รวม allowed_amount ของค่าใช้จ่ายที่ผูกกับ income
- อัปเดต income_type_id, type_name, amount แล้วคำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- ยอดเงินได้ใหม่ต้องไม่น้อยกว่าค่าใช้จ่ายที่หักได้เดิม
- ข้อมูลต่าง record หรือคนละเจ้าของตอบ 403

ผลลัพธ์

- บันทึก Income และยอดภาษีใหม่

20. destroyIncome(Request $request, TaxRecord $record, Income $income)

ตำแหน่ง	บรรทัด 65
Route	**DELETE /member/tax-records/{record}/incomes/{income}**
หน้าที	ลบรายการเงินได้

ขั้นตอนการทำงาน

- ตรวจเจ้าของและความสัมพันธ์
- ลบ Income
- เรียก calculateRecord()

การตรวจสอบ/ข้อจำกัด

- ข้อมูลต่าง record หรือคนละเจ้าของตอบ 403

ผลลัพธ์

- ลบรายการและอัปเดตผลรวม

21. storeExpense(Request $request, TaxRecord $record)

ตำแหน่ง	บรรทัด 73
Route	**POST /member/tax-records/{record}/expenses**
หน้าที	เพิ่มค่าใช้จ่ายให้กับรายการเงินได้

ขั้นตอนการทำงาน

- ตรวจ income_id, expense_type_id และ amount
- ตรวจว่า Income อยู่ใน record และ ExpenseType ตรงปี/ประเภทเงินได้/ยัง active
- ถ้า method เป็น flat ระบบคำนวณจาก income x rate; ถ้า actual ใช้ยอดที่กรอก
- จำกัด allowed_amount ด้วย max_amount, ยอด Income และวงเงินคงเหลือหลังหักค่าใช้จ่ายรายการอื่น
- สร้าง Expense แล้วคำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- amount อนุญาตให้ว่างและถือเป็น 0
- ประเภทค่าใช้จ่ายต้องตรงกับปีภาษีและประเภทเงินได้
- ผลรวมค่าใช้จ่ายที่ใช้ได้ต้องไม่เกินเงินได้รายการนั้น

ผลลัพธ์

- เพิ่ม Expense พร้อม amount และ allowed_amount ที่ระบบอนุญาต

22. updateExpense(Request $request, TaxRecord $record, Expense $expense)

ตำแหน่ง	บรรทัด 119
Route	**PUT /member/tax-records/{record}/expenses/{expense}**
หน้าที	แก้ไขยอดค่าใช้จ่ายเดิม

ขั้นตอนการทำงาน

- ตรวจเจ้าของผ่าน Expense -> Income -> TaxRecord
- อ่าน ExpenseType เดิม
- คำนวณยอดเหมาใหม่หรือใช้ยอดจริง
- ใช้เพดานประเภท รายได้ และวงเงินคงเหลือ โดยไม่นับ expense ปัจจุบันซ้ำ
- บันทึกและคำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- amount ต้องเป็นตัวเลขไม่ติดลบเมื่อมีค่า
- ข้อมูลต่าง record ตอบ 403

ผลลัพธ์

- อัปเดต amount และ allowed_amount

23. destroyExpense(Request $request, TaxRecord $record, Expense $expense)

ตำแหน่ง	บรรทัด 148
Route	**DELETE /member/tax-records/{record}/expenses/{expense}**
หน้าที	ลบรายการค่าใช้จ่าย

ขั้นตอนการทำงาน

- ตรวจเจ้าของและความสัมพันธ์
- ลบ Expense
- คำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- ข้อมูลต่าง record หรือคนละเจ้าของตอบ 403

ผลลัพธ์

- ลบ Expense และปรับยอดภาษี

24. storeDeduction(Request $request, TaxRecord $record)

ตำแหน่ง	บรรทัด 156
Route	**POST /member/tax-records/{record}/deductions**
หน้าที	เพิ่มรายการค่าลดหย่อน

ขั้นตอนการทำงาน

- ตรวจ deduction_type_id และ amount
- ตรวจ DeductionType ตรงปีและ active

- จำกัด allowed_amount ด้วย max_amount
- สร้าง Deduction แล้วคำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- amount ต้องเป็นตัวเลขไม่ติดลบ
- ประเภทต้องมีจริง ตรงปี และเปิดใช้งาน

ผลลัพธ์

- เพิ่ม Deduction โดยเก็บยอดกรอกและยอดที่อนุญาตแยกกัน

25. updateDeduction(Request $request, TaxRecord $record, Deduction $deduction)

ตำแหน่ง	บรรทัด 182
Route	**PUT /member/tax-records/{record}/deductions/{deduction}**
หน้าที	แก้ไขยอดค่าลดหย่อน

ขั้นตอนการทำงาน

- ตรวจเจ้าของและความสัมพันธ์
- อ่าน DeductionType เดิม
- จำกัด allowed_amount ตาม max_amount
- บันทึกและคำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- amount จำเป็น เป็นตัวเลข และไม่ติดลบ
- ข้อมูลต่าง record ตอบ 403

ผลลัพธ์

- อัปเดต Deduction และผลภาษี

26. destroyDeduction(Request $request, TaxRecord $record, Deduction $deduction)

ตำแหน่ง	บรรทัด 198
Route	**DELETE /member/tax-records/{record}/deductions/{deduction}**
หน้าที	ลบรายการค่าลดหย่อน

ขั้นตอนการทำงาน

- ตรวจเจ้าของและความสัมพันธ์
- ลบ Deduction
- คำนวณ record ใหม่

การตรวจสอบ/ข้อจำกัด

- ข้อมูลต่าง record หรือคนละเจ้าของตอบ 403

ผลลัพธ์

- ลบรายการและอัปเดตผลภาษี

27. calculateRecord(TaxRecord $record)

ตำแหน่ง	บรรทัด 207
Route	**ไม่มี Route โดยตรง - เรียกใช้ภายใน Controller**
หน้าที	รวมยอดและคำนวณภาษีใหม่หลังแก้รายการในประวัติ

ขั้นตอนการทำงาน

- รวม Income.amount และ Expense.allowed_amount
- แยก donation ออกจากค่าลดหย่อนอื่น
- หาฐานก่อนบริจาค = รายได้รวม - เงินได้ยกเว้น - ค่าใช้จ่าย - ลดหย่อนอื่น และไม่ให้ต่ำกว่า 0
- จำกัดเงินบริจาค 10% และกระจาย allowed_amount ให้รายการบริจาคตามลำดับ
- คำนวณเงินได้สุทธิและภาษีขั้นบันไดจาก tax_brackets ของปีใน record
- รวมเงินได้ที่ IncomeType.code ไม่ใช่ 40_1 เพื่อคำนวณภาษีขั้นต่ำ 0.5% เมื่อยอดตั้งแต่ 120,000 บาท
- ใช้ภาษีขั้นต่ำเมื่อมากกว่า 5,000 และมากกว่าภาษีขั้นบันได
- หัก withholding_tax และ half_year_tax แล้วเขียน additional_tax หรือ refund
- บันทึกยอดรวมทั้งหมดกลับ TaxRecord

การตรวจสอบ/ข้อจำกัด

- ใช้ข้อมูลอ้างอิงตามปีของ record
- ค่าฐานภาษีและเงินได้สุทธิถูกป้องกันไม่ให้ต่ำกว่า 0

ผลลัพธ์

- อัปเดต total_income, total_expense, total_deduction, donation_allowed, net_income, step_tax, minimum_tax, tax_payable, additional_tax และ refund

28. destroy(Request $request, TaxRecord $record)

ตำแหน่ง	บรรทัด 264
Route	**DELETE /member/tax-records/{record} | name: tax.records.destroy**
หน้าที	ลบประวัติผลคำนวณทั้งรายการ

ขั้นตอนการทำงาน

- ตรวจว่า record เป็นของผู้ใช้
- เรียก delete()
- redirect กลับหน้าประวัติ

การตรวจสอบ/ข้อจำกัด

- record ของผู้อื่นตอบ HTTP 403

ผลลัพธ์

- ลบ TaxRecord; รายการลูกจะเป็นไปตาม foreign key/cascade ที่กำหนดในฐานข้อมูล

AdminController

app/Http/Controllers/AdminController.php

หน้าที	Dashboard ผู้ดูแล จัดการผู้ใช้ และจัดการข้อมูลอ้างอิงทีสูตรภาษีใช้จากฐานข้อมูล ทุก Route อยู่ใต้ middleware auth + admin
จำนวนฟังก์ชัน	17

29. index()

ตำแหน่ง	บรรทัด 15
Route	**GET /admin/dashboard | name: admin.dashboard**
หน้าที	แสดงสถิติบน Dashboard ผู้ดูแล

ขั้นตอนการทำงาน

- นับผู้ใช้ทังหมด
- นับผู้ใช้ role admin
- นับ TaxRecord ทังหมด
- ส่ง 3 ยอดไป view admin.dashboard

ผลลัพธ์

- คืนหน้า Dashboard ผู้ดูแล

30. users()

ตำแหน่ง	บรรทัด 24
Route	**GET /admin/users | name: admin.users**
หน้าที	แสดงรายชือผู้ใช้

ขั้นตอนการทำงาน

- เรียง users ตาม id
- แบ่งหน้าละ 15 รายการ

ผลลัพธ์

- คืน view admin.users พร้อม paginator

31. updateRole(Request $request, User $user)

ตำแหน่ง	บรรทัด 30
Route	**PATCH /admin/users/{user}/role**
หน้าที	เปลียนสิทธิผู้ใช้ระหว่าง user และ admin

ขั้นตอนการทำงาน

- ตรวจค่า role
- ป้องกัน Admin เปลียนสิทธิของบัญชีตนเอง
- บันทึก role ใหม่

การตรวจสอบ/ข้อจำกัด

- role ต้องเป็น user หรือ admin
- ห้ามเปลียนสิทธิของบัญชีทีกำลังใช้งาน

ผลลัพธ์

- redirect ไป admin.users พร้อมข้อความสำเร็จ หรือส่ง validation error

32. updateUser(Request $request, User $user)

ตำแหน่ง	บรรทัด 51
Route	**PUT /admin/users/{user}**
หน้าที	แก้ชื่อและอีเมลของผู้ใช้

ขั้นตอนการทำงาน

- ตรวจชื่อและอีเมล
- ปรับอีเมลเป็นตัวพิมพ์เล็กและตัดช่องว่าง
- บันทึก User

การตรวจสอบ/ข้อจำกัด

- อีเมลต้องไม่ซ้ำ โดยยกเว้นผู้ใช้คนที่กำลังแก้

ผลลัพธ์

- กลับหน้ารายชื่อพร้อมข้อความสำเร็จ

33. destroyUser(Request $request, User $user)

ตำแหน่ง	บรรทัด 63
Route	**DELETE /admin/users/{user}**
หน้าที	ลบบัญชีผู้ใช้

ขั้นตอนการทำงาน

- ตรวจว่าเป้าหมายไม่ใช่บัญชี Admin ที่กำลังใช้งาน
- เรียก delete()

การตรวจสอบ/ข้อจำกัด

- ห้ามลบบัญชีตนเอง

ผลลัพธ์

- ลบ User และกลับหน้ารายชื่อพร้อมข้อความ

34. incomeTypes()

ตำแหน่ง	บรรทัด 72
Route	**GET /admin/income-types**
หน้าที	แสดงรายการประเภทเงินได้

ขั้นตอนการทำงาน

- อ่าน IncomeType ทั้งหมดเรียง code
- ส่งไป view admin.income-types

ผลลัพธ์

- คืนหน้าจัดการประเภทเงินได้

35. storeIncomeType(Request $request)

ตำแหน่ง	บรรทัด 78
Route	**POST /admin/income-types**
หน้าที	เพิ่มประเภทเงินได้

ขั้นตอนการทำงาน

- ตรวจ code, name, description
- สร้าง IncomeType และตั้ง is_active = true

การตรวจสอบ/ข้อจำกัด

- code จำเป็น ยาวไม่เกิน 30 และห้ามซ้ำ
- name จำเป็น ยาวไม่เกิน 255

ผลลัพธ์

- เพิ่มข้อมูล income_types

36. updateIncomeType(Request $request, IncomeType $incomeType)

ตำแหน่ง	บรรทัด 94
Route	**PUT /admin/income-types/{incomeType}**
หน้าที	แก้ชื่อ คำอธิบาย และสถานะประเภทเงินได้

ขั้นตอนการทำงาน

- ตรวจข้อมูล
- อัปเดต name, description, is_active

การตรวจสอบ/ข้อจำกัด

- is_active ต้องเป็น boolean

ผลลัพธ์

- อัปเดต income_types โดยไม่เปลี่ยน code

37. expenseTypes()

ตำแหน่ง	บรรทัด 108
Route	**GET /admin/expense-types**
หน้าที	แสดงประเภทค่าใช้จ่ายและประเภทเงินได้ที่สัมพันธ์กัน

ขั้นตอนการทำงาน

- eager load incomeType
- เรียง ExpenseType จากปีล่าสุดและ code
- อ่าน IncomeType สำหรับใช้ในฟอร์มเพิ่ม

ผลลัพธ์

- คืน view admin.expense-types

38. storeExpenseType(Request $request)

ตำแหน่ง	บรรทัด 115
Route	**POST /admin/expense-types**
หน้าที	เพิ่มกฎค่าใช้จ่ายของปีภาษี

ขั้นตอนการทำงาน

- ตรวจประเภทเงินได้ code ชื่อ ปี วิธี อัตรา เพดาน และคำอธิบาย
- ตรวจ code ไม่ซ้ำภายในปีเดียวกัน
- สร้าง ExpenseType และเปิดใช้งาน

การตรวจสอบ/ข้อจำกัด

- tax_year 2500-2600

- method เป็น flat หรือ actual
- rate 0-1
- max_amount ไม่ติดลบ

ผลลัพธ์

- เพิ่ม expense_types ที่สูตรคำนวณสามารถอ่านได้

39. updateExpenseType(Request $request, ExpenseType $expenseType)

ตำแหน่ง	บรรทัด 139
Route	**PUT /admin/expense-types/{expenseType}**
หน้าที่	แก้รายละเอียดอัตราค่าใช้จ่าย

ขั้นตอนการทำงาน

- ตรวจ name, rate, max_amount, description, is_active
- บันทึกค่าที่แก้

การตรวจสอบ/ข้อจำกัด

- rate ต้องอยู่ 0-1
- max_amount ไม่ติดลบ

ผลลัพธ์

- อัปเดต ExpenseType โดยไม่แก้ code, ปี และ income_type_id

40. deductionTypes()

ตำแหน่ง	บรรทัด 155
Route	**GET /admin/deduction-types**
หน้าที่	แสดงประเภทค่าลดหย่อน

ขั้นตอนการทำงาน

- อ่านทั้งหมด เรียงปีล่าสุดแล้วตาม code

ผลลัพธ์

- คืน view admin.deduction-types

41. storeDeductionType(Request $request)

ตำแหน่ง	บรรทัด 161
Route	**POST /admin/deduction-types**
หน้าที่	เพิ่มประเภทค่าลดหย่อนของปีภาษี

ขั้นตอนการทำงาน

- ตรวจข้อมูลและ code ไม่ซ้ำในปีเดียวกัน
- สร้าง DeductionType พร้อม is_active = true

การตรวจสอบ/ข้อจำกัด

- tax_year 2500-2600
- max_amount ว่างได้ แต่ถ้ามีต้องไม่ติดลบ

ผลลัพธ์

- เพิ่ม deduction_types

42. updateDeductionType(Request $request, DeductionType $deductionType)

ตำแหน่ง	บรรทัด 180
Route	**PUT /admin/deduction-types/{deductionType}**
หน้าที	แก้รายละเอียดและสถานะค่าลดหย่อน

ขั้นตอนการทำงาน

- ตรวจ name, max_amount, description, is_active
- บันทึกค่าใหม่

การตรวจสอบ/ข้อจำกัด

- is_active ต้องเป็น boolean
- max_amount ไม่ติดลบ

ผลลัพธ์

- อัปเดต DeductionType โดยไม่เปลี่ยน code และปี

43. taxBrackets()

ตำแหน่ง	บรรทัด 194
Route	**GET /admin/tax-brackets**
หน้าที	แสดงขั้นภาษี

ขั้นตอนการทำงาน

- อ่าน TaxBracket เรียงปีล่าสุดและ lower_bound

ผลลัพธ์

- คืน view admin.tax-brackets

44. storeTaxBracket(Request $request)

ตำแหน่ง	บรรทัด 200
Route	**POST /admin/tax-brackets**
หน้าที	เพิ่มขั้นภาษีของปีภาษี

ขั้นตอนการทำงาน

- ตรวจปี ขอบล่าง ขอบบน และอัตรา
- ตรวจ lower_bound ไม่ซ้ำในปีเดียวกัน
- สร้าง TaxBracket

การตรวจสอบ/ข้อจำกัด

- upper_bound ว่างได้สำหรับขั้นบนสุด หรือถ้ามีต้องมากกว่า lower_bound
- rate 0-1
- tax_year 2500-2600

ผลลัพธ์

- เพิ่ม tax_brackets ทีฟังก์ชันคำนวณใช้

45. updateTaxBracket(Request $request, TaxBracket $taxBracket)

ตำแหน่ง	บรรทัด 217
Route	**PUT /admin/tax-brackets/{taxBracket}**
หน้าที	แก้ขอบบนและอัตราของขั้นภาษี

ขั้นตอนการทำงาน

- ตรวจ upper_bound เทียบกับ lower_bound เดิม
- อัปเดต upper_bound และ rate

การตรวจสอบ/ข้อจำกัด

- upper_bound ต้องมากกว่า lower_bound หรือเป็น null
- rate ต้องอยู่ 0-1

ผลลัพธ์

- อัปเดตขันภาษี โดยไม่เปลี่ยนปีและขอบล่าง

ภาคผนวก: Middleware และคำสำคัญ

guest	อนุญาตเฉพาะผู้ที่ยังไม่ได้เข้าสู่ระบบ
auth	ต้องเข้าสู่ระบบแล้ว
member	อนุญาตส่วนสมาชิกตาม Middleware ของโปรเจกต์
admin	อนุญาตเฉพาะผู้ใช้ role admin
validate	ตรวจข้อมูลตามกฎ; ถ้าไม่ผ่าน Laravel จะย้อนกลับพร้อม errors
Route Model Binding	Laravel แปลง {record}, {user} หรือ id อื่นให้เป็น Model จากพารามิเตอร์ฟังก์ชัน
Flash Session	ข้อความ success/error ที่ใช้ชั่วคราวหลัง redirect
allowed_amount	ยอดค่าใช้จ่ายหรือค่าลดหย่อนที่ระบบยอมให้นำไปคำนวณ หลังใช้เพดานแล้ว
abort(403)	หยุดคำขอเมื่อผู้ใช้ไม่มีสิทธิ์เข้าถึงข้อมูลนั้น

ไฟล์ที่ใช้ตรวจสอบ

- app/Http/Controllers/*.php
- routes/web.php และ routes/settings.php